

Internship Report

B.Eng. in Software Engineering, Academic Year 2023

International College, King Mongkut's Institute of Technology Ladkrabang

By

64011752 Theint Nandarsu

Internship period: 22nd May – 14th July 2023

1. Company outline

1.1 Company name and address

Frontware International

1213/390, Town in Town, 94 Plubpla, Wang Thonglang, Bangkok 10310, Thailand

Tel. +6625592308

1.2 Main activities

Frontware International is a software outsourcing company and has provided software development solutions to customers around the world.

1. Mobile Application – develop native applications on Android and iOS with Flutter. Also develop PWA applications such as PromptPay QRcode that helps small market shops and SME merchants to generate QRcode and share it on mobile phone.
<https://promptpay.frontware.co.th/profiles>
2. Odoo – supports customers to integrate Odoo in their businesses.
3. Weladee – Weladee is a Time Attendance System that collects the information of working presence of employees. Employees can record time attendance from mobile phone (Android and iPhone). Directors and managers get a clear overview of real time attendance from an online dashboard.

The main projects that are currently ongoing are Fluxcore application, Weladee and PromptPay Generator.

1.3 Organization structure

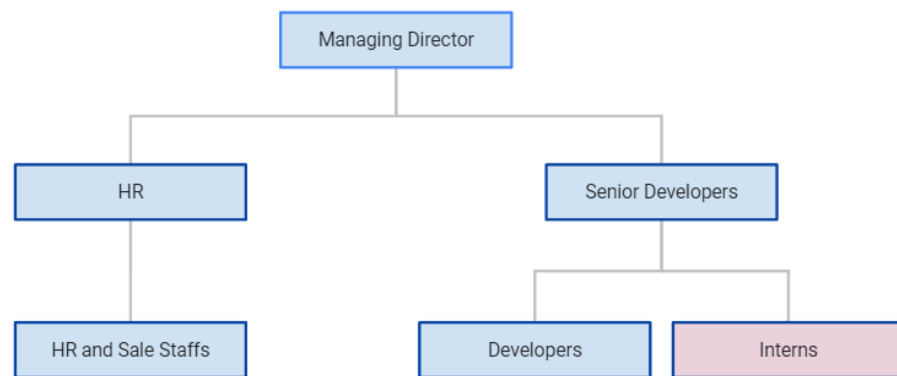
Developer Team – Develop the mobile application, web, system administration.

Current Projects – PromptPay Generator, Fluxcore Application, Weladee

(Estimated – 10 people)

HR Team – manage the employees' information and payroll, benefits management, recruiting, onboarding and etc.

(Estimated – 3 people)



1.4 Department and supervision

Main supervisor – Mr.Erci Fairon, Mr. Kriangchai Pongkittisopa

Tel: +66 8 6789 7745

Software Development Team

Project Team Leader – Mr. Florian Sabate

Software Development Team

Fluxcore Application Development

2. Internship

2.1 Original description and objectives

The objective of this internship is to build the dashboard with Flutter for Fluxcore application with the team together. The dashboard shows detailed information of computers, their geographic locations, CPU, GPU, RAM, and active timestamps. The main tasks I am a part of are to display the data with pie chart and tables, making the dashboard visually appealing.

2.2 Revised description and objectives

The main task of the internship is the same (building the dashboard with Flutter) but I have to change the project to Flux core from PingPrice as PingPrice project has not been initiated yet. Another thing is about testing benchmarks from MLCommons. As the work needs higher servers and GPU requirements, the process is in hiatus, and I have been assigned to build the Flux core admin dashboard panel with Flutter.

3. Software Development

As part of my internship, my assigned task was to develop an admin dashboard panel using Flutter. The tasks included implementing the following key functionalities:

1. Data Visualization:

- Create a pie chart to visually represent the data related to computer details such as CPU, RAM, storage, GPU, and network status.
- Utilize Flutter's charting libraries, such as charts_flutter, to generate interactive and informative visualizations.

2. Navigation Bar and Menus:

- Design and integrate a navigation bar into the admin dashboard panel, allowing easy access to different sections and functionalities.
- Add menus to the navigation bar for seamless navigation between various computer details and system management options.

3. Computer Details Display:

- Retrieve and display detailed information about the computer, including CPU utilization, RAM usage, storage capacity, GPU status, and network connectivity.
- Use Flutter's widget system to create dynamic and responsive interfaces that present the information clearly and concisely.

4. Real-time Updates:

- Implement mechanisms to fetch and display real-time updates for computer metrics, ensuring administrators have access to the latest information.
- Utilize Flutter's async programming capabilities and integrate with appropriate APIs or backend systems to retrieve and update data dynamically.

6. Testing and Debugging:

- Conduct comprehensive testing to ensure the functionality and reliability of the admin dashboard panel.

By successfully implementing these tasks, the admin dashboard panel will provide administrators with a comprehensive overview of computer details, enabling efficient monitoring and management of system resources.

3.1 Background

The developed software is an admin dashboard panel created using Flutter for the RunOnFlux project, a platform designed to streamline and enhance the management and monitoring of running events. The admin dashboard panel serves as a vital component of the RunOnFlux system, providing administrators with a comprehensive overview of computer details and system metrics during events.

1. Purpose of Software Development:

The software was developed to empower event organizers and administrators with a user-friendly interface to efficiently manage and monitor the Runonflux system during running

events. By offering real-time insights into computer details such as CPU, RAM, storage, GPU, and network status, the admin dashboard panel enables event organizers to make informed decisions and ensure a smooth event experience for participants and spectators.

2. Improvement of Existing System:

Prior to the development of the admin dashboard panel, the Runonflux project lacked a centralized and intuitive interface for administrators to monitor computer details and system status during events. Existing systems may have relied on separate tools or manual monitoring, leading to inefficiencies and difficulties in managing large-scale events effectively.

3. Interaction with Larger System:

The admin dashboard panel is a crucial part of the larger Runonflux system, which encompasses various modules like event registration, ticketing, and data analytics. The admin panel interacts with backend services to fetch real-time data about computers and communicates with other modules to provide event organizers with relevant information.

In conclusion, the development of the admin dashboard panel for the Runonflux project fills a crucial gap in the system, enabling event organizers to efficiently manage and monitor computer details during running events. By creating a customized and integrated solution, the admin dashboard panel optimizes event management and enhances the overall Runonflux experience for all stakeholders involved.

3.2 Software requirement specification

User Functional Requirements

- The system shall let the user view the status of the computer list.

- The system shall let the user view the detailed information of each computer.

- The system shall allow the user to filter the data and view only what they want to view.

User Non-Functional Requirements

- The system shall respond quickly and effectively to the user interactions.

- The system shall display clear error messages and instructions.

- The system shall have an intuitive and user-friendly interface.

System Functional Requirements

- The system shall integrate with various data sources to retrieve real-time data computer details during events.

- The system shall be responsive to different devices.

System Non-Functional Requirements

- The system shall be developed in Flutter 3.10

- The system shall use GitLab for version control.

- The system shall be developed on Linux OS.

3.3 Development team

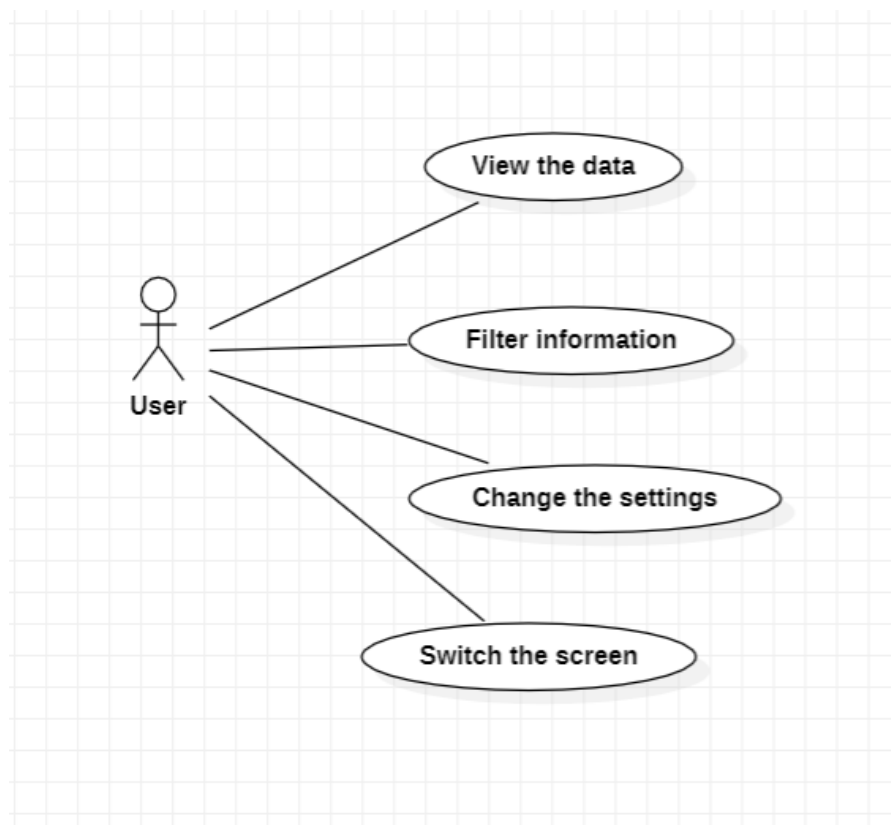
1. Mr. Alex Fairon
Responsible: Benchmark Testing and Extracting data
2. Mr. Florian Sabate
Responsible: Developing Flux core Application with Flutter
3. Mr. Arthur Page
Responsible: Developing Flux core Application with Flutter
4. Miss Theint Nandar Su
Responsible: Developing Flux core Application with Flutter

3.4 Project plan

The plan for the Flux core Application team is to develop the PWA application with Flutter. There are two teams that work on front-end and back-end separately. The backend team must find and extract the data. The front-end team must make the dashboard visually appealing by using Flutter. I am assigned to make the main screen of the dashboard and its side screens with the data.

3.5 Software design

The admin dashboard panel for the RunOnFlux software was designed as a modular and scalable application, developed using the Flutter framework. The main goal of the software was to provide event organizers with real-time insights into computer details during running events. The dashboard panel served as a front-end component, interacting with backend services to fetch data and display it in a user-friendly manner.



3.6 Implementation

3.6.1 Development tools

Tools:

- Dart
- Golang
- Python 3.6
- Flutter Framework
- GitLab

Libraries, Frameworks, and IDE

- Flutter
- TensorFlow
- Visual Studio Code
- CUDA
- cuDNN
- Pytorch
- Numpy
- Scipy
- Pandas

3.6.2 Techniques

Techniques for creating responsive user interfaces were applied to ensure the admin dashboard panel displayed correctly on different screen sizes and orientations. Flutter's responsive widgets and media query handling ensured a consistent user experience across various devices.

Algorithms for sorting and filtering data were employed to enable users to view computer details based on their preferences. Sorting and filtering functionalities improved data accessibility and made it easier for administrators to find specific information.

3.7 Testing

The way we test our dashboard display is first to add the necessary functions and libraries to make the dashboard visually appealing. When we have implemented the functions, we need to debug the errors and revise the work until the expected output has been released. After our supervisor has checked the code flow and its output, we add our version to the main project.

3.8 Screenshots

4. Achievement

4.1 Summary of work done

. During the internship, my main task was to create an admin dashboard panel using Flutter for the RunOnFlux project. I have to make the dashboard visually appealing and add the functions to calculate the data and display them with pie charts.

Estimated Lines of Codes

4.2 Evaluation

As I evaluate my internship objectives, I am pleased to say that I have achieved a significant level of success. During the internship, my main task was to create an admin dashboard panel using Flutter for the RunOnFlux project. I can confidently state that I accomplished most of the assigned objectives within the given timeframe.

The admin dashboard panel effectively fulfilled its core functionalities, providing an intuitive and interactive interface for users to monitor and manage computer details. I have gained valuable experience and improved my skills in Flutter, Golang, data visualization, and real-time data handling. Moving forward, I will use the lessons learned from this internship to approach future projects with increased efficiency and a focus on user experience. Additionally, I will continue to enhance my skills in performance optimization and documentation, recognizing their importance in delivering high-quality software solutions.

5. Conclusion

5.1 Skills and knowledge learned

During my internship, the skills and knowledge I gained from studying Software Engineering played a significant role in my work, making the experience both productive and rewarding. My understanding of languages like Flutter and Golang allowed me to develop the admin dashboard panel effectively. I improved my proficiency in Flutter significantly. Working extensively with these technologies enhanced my confidence in developing mobile applications and backend services. I also learned how to create interactive data visualizations using Flutter's charting libraries, which was a new skill that added value to the dashboard's presentation of computer details.

The most important lesson I will take forward from my internship is the value of teamwork and collaboration. Working alongside experienced professionals and being part of a supportive team boosted my learning and productivity.

Materials:

- Computer Organization
- Object-oriented Programming
- User-Experience Design

5.2 Problems and possible improvement

During my internship, I encountered a few challenges and problems that required problem-solving and adaptation. Some of the notable difficulties I faced include:

1. Responsive Design:

Designing a responsive and visually appealing interface for the admin dashboard panel was another challenge. Ensuring that the dashboard displayed properly on various screen sizes,

orientations, and devices required careful consideration of layout, spacing, and widget responsiveness. Iterative testing and adjustments were necessary to achieve a seamless user experience.

2. Debugging and Troubleshooting:

During the development process, I encountered occasional bugs and errors that impacted the functionality of the admin dashboard panel. Debugging and troubleshooting these issues required thorough analysis, meticulous testing, and collaboration with team members. Identifying the root causes and implementing effective solutions demanded a systematic approach.

Overall, these challenges provided valuable learning opportunities and helped me develop problem-solving skills, adaptability, and resilience as a software developer. Through perseverance and collaboration, I was able to overcome these obstacles and contribute effectively to the development of the admin dashboard panel.

6. Acknowledgement

I wanted to take a moment to express my heartfelt gratitude to each colleague for unwavering support and guidance throughout this journey.

The mentorship of CEO and supervisor has been invaluable, and I am deeply thankful for the opportunity to learn and grow under their leadership. Their patience and willingness to answer my questions, provide feedback, and share their expertise have been instrumental in my development as a software developer. I am also thankful to my team members for welcoming me into the team with open arms and for fostering a collaborative and positive environment.

I am grateful for the trust and responsibility you placed in me during the internship, allowing me to contribute to the development of the admin dashboard panel for the RunOnFlux project. The challenges I encountered along the way have taught me resilience and problem-solving skills, and I appreciate the opportunities to learn and adapt.

Once again, thank you for making this internship a truly memorable and rewarding experience. Their guidance and mentorship will undoubtedly shape my career journey positively.

7. Reference

1. <https://medium.com/analytics-vidhya/mlperf-getting-your-feet-wet-with-benchmarking-ml-workloads-6ecd57620ac6>
2. https://gitlab.com/frontware_International/protech/qrcode/-/blob/develop/qrcode/lib/widgets/field_machine.dart
3. https://gitlab.com/frontware_International/protech/qrcode/-/issues/18
4. https://gitlab.com/frontware_International/runonflux/pouwfrontend/-/blob/develop/flutter/lib/router/router.dart

8. Source Code

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_admin/blocs/cubit/cities_cubit.dart';
import 'package:flutter_admin/blocs/cubit/parquet_export_cubit.dart';
import 'package:flutter_admin/data/extensions/computer_request.dart';
import 'package:flutter_admin/data/extensions/translation_extension.dart';
import 'package:flutter_admin/data/models/computer_filter.dart';
import 'package:flutter_admin/proto/admin.pb.dart';
import 'package:flutter_admin/widgets/dialog.dart';
import 'package:flutter_admin/widgets/header_filter.dart';
import 'package:flutter_admin/widgets/search_bar.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_svg/svg.dart';
import 'package:go_router/go_router.dart';
import 'package:url_launcher/url_launcher.dart';
import '../blocs/computer_filter.dart';
import '../blocs/computer_list_cubit.dart';
import '../data/grpc_cubit.dart';
import '../widgets/computer_table_widget.dart';

class ComputerListScreen extends StatelessWidget {
  const ComputerListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (context) => ComputerFilterCubit()),
        BlocProvider(create: (context) => CitiesCubit()),
      ],
      child: Builder(builder: (context) {
        return BlocListener<ComputerFilterCubit, ComputerFilterState>(
          listener: (context, state) {
            context
              .read<CitiesCubit>()
              .get(state.computerRequest.realCountry);
          },
          listenWhen: (previous, current) {
            return previous.country != current.country;
          },
          child: Scaffold(
            drawerScrimColor: Colors.black45,
            appBar: AppBar(
              toolbarHeight: 100,
              backgroundColor: Colors.grey[100],
              leading: Builder(
                builder: (context) => IconButton(
                  icon: const Icon(Icons.filter_list),
                ),
                IconButton(
                  onPressed: () {
                    context.go("/benchmark");
                  }
                )
              )
            )
          )
        )
      )
    );
  }
}
```

```

        context.go('/home');
      },
      icon: const Icon(Icons.add)),
      icon: const Icon(
        Icons.home_filled,
      )),
      IconButton(
        onPressed: () {
          context.read<ComputerListCubit>().refresh();
          icon: const Icon(Icons.refresh)),
      IconButton(
        onPressed: () {
          context.go("/settings");
          context.go('/settings');
        },
        icon: const Icon(
          Icons.settings,
        drawer: const HeaderFilter(),
        body: const Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [ComputerTable()],
          children: [
            ComputerTable(),
          ],
        ),
      ),
    ),
  );

```

```

import 'package:flutter/material.dart';
import 'package:flutter_admin/blocs/computer_list_cubit.dart';
import 'package:flutter_admin/data/extensions/computer_extension.dart';
import 'package:flutter_admin/data/extensions/iterable_extension.dart';
import 'package:flutter_admin/data/grpc_cubit.dart';
import 'package:flutter_admin/proto/hardware/computer.pb.dart';
import 'package:flutter_admin/widgets/pie.dart';
import 'package:flutter_admin/widgets/simple_container.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:pie_chart/pie_chart.dart';

```

```

class DataPie2 extends StatefulWidget {
  const DataPie2({super.key});

  @override
  State<DataPie2> createState() => _DataPie2State();
}

```

```

class _DataPie2State extends State<DataPie2> {
  ComputeData? _computeData;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {

```

```

        context.read<ComputerListCubit>().defaultGet();
    });
}

@override
Widget build(BuildContext context) {
    return BlocListener<ComputerListCubit, GrpcState<List<Computer>>>>(
        listener: (context, state) {
            if (state.isFinished() && state.hasData() && mounted) {
                setState(() {
                    _computeData = ComputeData(state.data!);
                });
            }
        },
        child: _build());
}

Widget _build() {
    if (_computeData == null) {
        return const SizedBox.shrink();
    }
    return SingleChildScrollView(
        child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
                Row(
                    children: [
                        Expanded(
                            child: SimpleContainer(
                                child: Column(
                                    children: [
                                        const Text(
                                            "Active and Inactive Networks Status",
                                            style: TextStyle(
                                                fontWeight: FontWeight.bold,
                                                fontSize: 20,
                                            ),
                                        ),
                                    ],
                                ),
                                const SizedBox(
                                    height: 20,
                                ),
                                MyPie(
                                    dataMap: _computeData!.networks,
                                    animationDuration: const Duration(milliseconds: 1),
                                    chartLegendSpacing: 30,
                                    chartRadius: MediaQuery.of(context).size.width / 7,
                                    initialAngleInDegree: 0,
                                    chartType: ChartType.disc,
                                    ringStrokeWidth: 32,
                                    legendOptions: const LegendOptions(
                                        showLegendsInRow: false,
                                        legendPosition: LegendPosition.right,
                                        showLegends: true,

```

```

        legendShape: BoxShape.circle,
        legendTextStyle: TextStyle(
          fontWeight: FontWeight.bold,
        ),
      ),
    chartValuesOptions: const ChartValuesOptions(
      showChartValueBackground: true,
      showChartValues: true,
      showChartValuesInPercentage: true,
      showChartValuesOutside: false,
      decimalPlaces: 1,
    ),
  ),
],
)),
),
const SizedBox(width: 30),
Expanded(
  child: SimpleContainer(
    child: Column(
      children: [
        const Text(
          "Locations of Users",
          style:
            TextStyle(fontWeight: FontWeight.bold, fontSize:
20),
        ),
      ],
    ),
  ),
  const SizedBox(
    height: 20,
  ),
  MyPie(
    dataMap: _computeData!.countries,
    animationDuration: const Duration(milliseconds: 1),
    chartLegendSpacing: 30,
    chartRadius: MediaQuery.of(context).size.width / 7,
    initialAngleInDegree: 0,
    chartType: ChartType.disc,
    ringStrokeWidth: 32,
    legendOptions: const LegendOptions(
      showLegendsInRow: false,
      legendPosition: LegendPosition.right,
      showLegends: true,
      legendShape: BoxShape.circle,
      legendTextStyle: TextStyle(
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
  chartValuesOptions: const ChartValuesOptions(
    showChartValueBackground: true,
    showChartValues: true,
    showChartValuesInPercentage: true,
    showChartValuesOutside: false,
    decimalPlaces: 1,
  ),
),

```

```

        ),
    ),
],
)),
),
],
),
const SizedBox(width: 50),
],
),
);
}
}

```

```

class ComputeData {
  final List<Computer> data;
  late Map<String, double> countries;
  late Map<String, double> networks;

  ComputeData(this.data) {
    countries = computeCountryDataMap(data);
    networks = computeNetworks(data);
  }

  Map<String, double> computeCountryDataMap(List<Computer> data, [int?
take]) {
    Map<String, double> dataMap1 = {};

    for (final entry
      in data.groupBy((p0) => p0.geoLocation.countryLong).entries) {
      final key = entry.key.isEmpty ? "N/A" : entry.key;
      final value = entry.value.length.toDouble();
      dataMap1[key] = value;
    }

    return _compute(dataMap1);
  }

  Map<String, double> computeNetworks(List<Computer> data) {
    Map<String, double> dataMap2 = {};

    for (final entry
      in data.groupBy((p0) => p0.onlineStatus.status.toString()).entries)
    {
      final key = entry.key;
      final value = entry.value.length.toDouble();
      dataMap2[key] = value;
    }

    return _compute(dataMap2);
  }

  Map<String, double> _compute(Map<String, double> data, [int? sliceNum]) {

```

```

    if (data.keys.length < 5) {
      return data;
    } else {
      sliceNum = 5;
      final sortedEntries = data.entries.toList()
        ..sort((a, b) => b.value.compareTo(a.value));

      final topEntries = sortedEntries.take(sliceNum).toList();
      final restValue = sortedEntries
        .skip(sliceNum)
        .map((entry) => entry.value)
        .reduce((value, element) => value + element);

      data.clear();
      for (final entry in topEntries) {
        data[entry.key] = entry.value;
      }

      data['Others'] = restValue;
      return data;
    }
  }
}

import 'package:flutter/material.dart';
import 'package:flutter_admin/blocs/computer_list_cubit.dart';
import 'package:flutter_admin/data/extensions/iterable_extension.dart';
import 'package:flutter_admin/data/grpc_cubit.dart';
import 'package:flutter_admin/proto/hardware/computer.pb.dart';
import 'package:flutter_admin/widgets/pie.dart';
import 'package:flutter_admin/widgets/simple_container.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:pie_chart/pie_chart.dart';

class DataPie1 extends StatefulWidget {
  const DataPie1({super.key});

  @override
  State<DataPie1> createState() => _DataPie1State();
}

class _DataPie1State extends State<DataPie1> {
  ComputeData? _computeData;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      context.read<ComputerListCubit>().defaultGet();
    });
  }

  @override

```

```

Widget build(BuildContext context) {
  return BlocListener<ComputerListCubit, GrpcState<List<Computer>>>>(
    listener: (context, state) {
      if (state.isFinished() && state.hasData() && mounted) {
        setState(() {
          _computeData = ComputeData(state.data!);
        });
      }
    },
    child: _build());
}

Widget _build() {
  if (_computeData == null) {
    return const SizedBox.shrink();
  }
  return SingleChildScrollView(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Row(
          children: [
            Expanded(
              child: SimpleContainer(
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Text(
                      "Types of OS",
                      style:
                        TextStyle(fontWeight: FontWeight.bold, fontSize:
20),
                    ),
                    const SizedBox(
                      height: 20,
                    ),
                    MyPie(
                      dataMap: _computeData!.osType,
                      animationDuration: const Duration(milliseconds: 1),
                      chartLegendSpacing: 30,
                      chartRadius: MediaQuery.of(context).size.width / 7,
                      initialAngleInDegree: 0,
                      chartType: ChartType.disc,
                      ringStrokeWidth: 32,
                      legendOptions: const LegendOptions(
                        showLegendsInRow: false,
                        legendPosition: LegendPosition.right,
                        showLegends: true,
                        legendShape: BoxShape.circle,
                        legendTextStyle: TextStyle(
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                    ),

```

```

        chartValuesOptions: const ChartValuesOptions(
            showChartValueBackground: true,
            showChartValues: true,
            showChartValuesInPercentage: true,
            showChartValuesOutside: false,
            decimalPlaces: 1,
        ),
    ),
],
)),
),
],
),
const SizedBox(width: 50),
],
),
);
}
}

class ComputeData {
    final List<Computer> data;

    late Map<String, double> osType;

    ComputeData(this.data) {
        osType = computeOsDataMap(data);
    }

    Map<String, double> computeOsDataMap(List<Computer> data) {
        Map<String, double> dataMap = {};

        for (final entry in data.groupBy((p0) => p0.os.oSType.name).entries) {
            dataMap[entry.key] = entry.value.length.toDouble();
        }
        return _compute(dataMap);
    }

    Map<String, double> _compute(Map<String, double> data, [int? sliceNum]) {
        if (data.keys.length < 5) {
            return data;
        } else {
            sliceNum = 5;
            final sortedEntries = data.entries.toList()
                ..sort((a, b) => b.value.compareTo(a.value));

            final topEntries = sortedEntries.take(sliceNum).toList();
            final restValue = sortedEntries
                .skip(sliceNum)
                .map((entry) => entry.value)
                .reduce((value, element) => value + element);

            data.clear();

```



```

    for (final entry in topEntries) {
      data[entry.key] = entry.value;
    }

    data['Others'] = restValue;
    return data;
  }
}

import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

class DashBoardMenu extends StatelessWidget {
  const DashBoardMenu({super.key});

  @override
  Widget build(BuildContext context) {
    return Drawer(
      child: ListView(
        padding: EdgeInsets.zero,
        children: [
          const DrawerHeader(
            margin: EdgeInsets.zero,
            padding: EdgeInsets.zero,
            decoration: BoxDecoration(
              image: DecorationImage(
                fit: BoxFit.fill,
                image: AssetImage(
                  'assets/images/png/sign_in_background.png'
                )),
            child: Stack(
              children: [
                Positioned(
                  bottom: 12.0,
                  left: 16.0,
                  child: Text(
                    'Fluxcore Admin',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 20.0,
                      fontWeight: FontWeight.w500),
                  ),
                ),
              ],
            )),
          ListTile(
            leading: const Icon(Icons.dashboard),
            title: const Text("Dashboard"),
            onTap: () {
              context.go('/home');
            },
          ),
          ListTile(
            leading: const Icon(Icons.list_alt),

```

```

        title: const Text("Computer List"),
        onTap: () {
          context.go('/computer_list_screen');
        },
      ),
      ListTile(
        leading: const Icon(Icons.settings),
        title: const Text("Settings"),
        onTap: () {
          context.go('/settings');
        },
      ),
    ],
  ),
);
}
}

import 'package:flutter/material.dart';
import 'package:flutter_admin/data/extensions/computer_extension.dart';
import 'package:flutter_admin/data/extensions/ram_extension.dart';
import 'package:flutter_admin/data/extensions/theme_manager.dart';
import 'package:flutter_admin/data/extensions/translation_extension.dart';
import 'package:flutter_admin/widgets/simple_container.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:go_router/go_router.dart';
import '../blocs/computer_list_cubit.dart';
import '../data/grpc_cubit.dart';
import '../proto/hardware/computer.pb.dart';

class ActiveComputer extends StatelessWidget {
  const ActiveComputer({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<ComputerListCubit, GrpcState<List<Computer>>>>(
      builder: (context, computerListState) {
        if (computerListState.hasData()) {
          final computerList =
            List<Computer>.from(computerListState.data!).take(4);
          List<DataColumn> columns = [
            const DataColumn(label: Text('Platform')),
            const DataColumn(label: Text('RAM')),
            const DataColumn(label: Text('Free Storage')),
            const DataColumn(label: Text('Location')),
            const DataColumn(label: Text('Status')),
          ];

          List<DataRow> rows = [];
          for (var i = 0; i < computerList.length; i++) {
            rows.add(DataRow(
              cells: [

```

```

                DataCell(Text(computerList.ele-
mentAt(i).os.oStype.toString()))),
                DataCell(
                    Text("${computerList.elementAt(i).rams.totalCapacity}
Gb")),
                DataCell(Text(
                    "${computerList.elementAt(i).storages.maxFreeSpace}
Gb")),
                DataCell(Text(
                    "${computerList.elementAt(i).geoLocation.countryLong}
${computerList.elementAt(i).geoLocation.city}")),
                DataCell(Text(computerList
                    .elementAt(i)
                    .onlineStatus
                    .status
                    .name
                    .toString()))),
            ],
        ));
    }
    return SingleChildScrollView(
        child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
                Row(
                    children: [
                        Expanded(
                            child: SimpleContainer(
                                child: Column(
                                    mainAxisAlignment: MainAxisAlignment.center,
                                    children: [
                                        const Text(
                                            "Sample Computer List",
                                            style: TextStyle(
                                                fontWeight: FontWeight.bold, fontSize: 20),
                                        ),
                                        const SizedBox(
                                            height: 20,
                                        ),
                                        DataTable(
                                            showCheckboxColumn: false,
                                            showBottomBorder: true,
                                            headingRowColor: MaterialStatePropertyAll(
                                                context.watchTheme.data.primaryColorLight),
                                            columns: columns,
                                            rows: rows,
                                        ),
                                        const SizedBox(
                                            height: 10,
                                        ),
                                        ElevatedButton(
                                            onPressed: () {
                                                context.go('/computer_list_screen');
                                            },

```

```

        },
        child: const Text("Tap for complete list",
            style: TextStyle(color: Colors.indigo)),
    ),
  ],
),
),
),
],
),
const SizedBox(width: 50),
],
),
);
} else {
  if (computerListState.isLoading()) {
    return const Center(child: CircularProgressIndicator());
  }
  if (!computerListState.hasData()) {
    return const Column(
      children: [
        Icon(Icons.warning),
        Center(
          child: Text("NO DATA"),
        ),
      ],
    );
  }
  return Center(
    child: Column(
      children: [
        const Icon(Icons.warning),
        Text(context.tr.failed),
        TextButton(
          onPressed: () {
            context.read<ComputerListCubit>().refresh();
          },
          child: Text(context.tr.tryAgain)
        ),
      ],
    ),
  );
}
});
}
}

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_admin/blocs/cubit/cities_cubit.dart';
import 'package:flutter_admin/blocs/cubit/parquet_export_cubit.dart';
import 'package:flutter_admin/data/extensions/computer_request.dart';
import 'package:flutter_admin/data/extensions/translation_extension.dart';

```

```

import 'package:flutter_admin/data/models/computer_filter.dart';
import 'package:flutter_admin/proto/admin.pb.dart';
import 'package:flutter_admin/widgets/datapie1.dart';
import 'package:flutter_admin/widgets/datapie2.dart';
import 'package:flutter_admin/widgets/active_computer_list.dart';
import 'package:flutter_admin/widgets/dashboard_menu.dart';
import 'package:flutter_admin/widgets/dialog.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_svg/svg.dart';

import 'package:url_launcher/url_launcher.dart';
import '../blocs/computer_filter.dart';
import '../data/grpc_cubit.dart';

class WelcomeScreen extends StatelessWidget {
  const WelcomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (context) => ComputerFilterCubit()),
        BlocProvider(create: (context) => CitiesCubit()),
      ],
      child: Builder(builder: (context) {
        return BlocListener<ComputerFilterCubit, ComputerFilterState>(
          listener: (context, state) {
            context
              .read<CitiesCubit>()
              .get(state.computerRequest.realCountry);
          },
          listenWhen: (previous, current) {
            return previous.country != current.country;
          },
          child: Scaffold(
            drawerScrimColor: Colors.black45,
            appBar: AppBar(
              toolbarHeight: 100,
              backgroundColor: Colors.grey[100],
              leading: Builder(
                builder: (context) => IconButton(
                  icon: const Icon(Icons.menu),
                  onPressed: () => Scaffold.of(context).openDrawer(),
                ),
              ),
              title: Row(
                children: [
                  const Text("Dashboard"),
                  SizedBox(
                    width: MediaQuery.sizeOf(context).width / 10,
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}

```

```

actions: [
  IconButton(
    onPressed: () {
      context.read<ParquetExportCubit>().get(Empty());
      CustomDialog(
        title: context.tr.parquetFile,
        body: exportToParquet()
      ).showAsync(context);
    },
    icon: SvgPicture.asset(
      'assets/images/svg/apache_parquet_icon.svg',
      width: 120,
    ),
  ),
],
),
drawer: const DashBoardMenu(),
body: const SingleChildScrollView(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Text(
        "Welcome to Fluxcore!",
        style: TextStyle(
          fontSize: 45,
          fontWeight: FontWeight.bold,
          color: Colors.indigo,
        ),
      ),
      SizedBox(
        height: 15,
      ),
      Row(
        children: [
          Expanded(child: DataPie1()),
          SizedBox(
            width: 30,
          ),
          Expanded(child: ActiveComputer()),
        ],
      ),
      SizedBox(
        height: 30,
      ),
      Row(
        children: [
          Expanded(child: DataPie2()),
        ],
      ),
    ],
  ),
),
)),
);

```

```

    ));
}

Widget exportToParquet() {
  return BlocBuilder<ParquetExportCubit, GrpcState<ParquetFileName>>(
    builder: (context, state) {
      if (state.isLoading()) {
        return const CircularProgressIndicator();
      } else if (state.isFinished() && !state.hasData()) {
        return Text(context.tr.noDataToConvert);
      } else if (state.isFinished() && state.hasData()) {
        return Column(
          children: [
            ElevatedButton(
              child: Text(context.tr.downloadParquetFile),
              onPressed: () {
                launchUrl(Uri.parse(state.data!.uRL));
              },
            ),
            Row(
              children: [
                Text("${context.tr.password} :"),
                const SizedBox(
                  width: 20,
                ),
                Text(state.data!.password),
                const SizedBox(
                  width: 20,
                ),
                IconButton(
                  onPressed: () {
                    Clipboard.setData(
                      ClipboardData(text: state.data!.password));
                    ScaffoldMessenger.of(context).showSnackBar(
                      const SnackBar(
                        content: Text("Copied to clipboard !")));
                  },
                  icon: const Icon(Icons.copy))
              ],
            ),
          ],
        );
      } else if (state.isIdle()) {
        return CustomDialog(
          title: context.tr.error,
          message: context.tr.noDataToConvert,
        );
      }
      return const SizedBox.shrink();
    },
  );
}

```

```

Future<void> _launchUrl(_url) async {

```

```

        if (!await launchUrl(_url)) {
            throw Exception('Could not launch $_url');
        }
    }
}

```

Changing the drop-down

```

ThemeData get _lightTheme => ThemeData(
  dropdownMenuTheme: DropdownMenuThemeData(
    menuStyle: const MenuStyle(
      elevation: MaterialStatePropertyAll(2),
      backgroundColor: MaterialStatePropertyAll(Color(0xFFEBEEFF)),
    ),
    inputDecorationTheme: InputDecorationTheme(
      border: OutlineInputBorder(
        borderSide: const BorderSide(
          color: Color.fromRGBO(184, 195, 225, 0.5), width: 1),
        borderRadius: BorderRadius.circular(10)),
      ),
    menuStyle: const MenuStyle(
      elevation: MaterialStatePropertyAll(2),
      backgroundColor:
        MaterialStatePropertyAll(Color.fromARGB(255, 164, 171, 207)),
    ),
    inputDecorationTheme: InputDecorationTheme(
      border: OutlineInputBorder(
        borderSide: const BorderSide(
          color: Color.fromRGBO(184, 195, 225, 0.5), width: 1),
        borderRadius: BorderRadius.circular(10)),
      ),
    ),
  ),
  colorScheme: ColorScheme.fromSeed(
    surface: Colors.white,
    surfaceTint: Colors.transparent,
    return previous.osType != current.osType;
  ),
  builder: (context, state) {
    return DropdownButton<OSType>(
      isExpanded: true,
      hint: Text(context.tr.osType),
      value: filterState.osType,
      items: items,
      onChanged: (OSType? newValue) {
        if (newValue != null) {
          context.read<ComputerFilterCubit>().updateOs(newValue);
        }
      },
    );
    return DecoratedBox(
      decoration: BoxDecoration(
        color: const Color.fromARGB(255, 224, 232, 245),
        border:
          Border.all(color: context.watchTheme.data.primaryCol-
orDark),
        borderRadius: BorderRadius.circular(10),

```



```

    ),
    child: Padding(
      padding: const EdgeInsets.only(left: 10, right: 10),
      child: DropdownButtonHideUnderline(
        child: DropdownButton<OSType>(
          isExpanded: true,
          hint: Text(context.tr.osType),
          value: filterState.osType,
          items: items,
          focusColor: Colors.transparent,
          icon: const Icon(Icons.keyboard_ar-
row_down_rounded),
          iconEnabledColor:
            context.watchTheme.data.primaryColorDark,
          onChanged: (OSType? newValue) {
            if (newValue != null) {
              context
                .read<ComputerFilterCubit>()
                .updateOs(newValue);
            }
          }
        )
      ),
    ),
  );
}

if (state.isLoading()) {
  return const Center(child: CircularProgressIndicator());
} else if (state.hasData()) {
  return DropdownButton<String>(
    isExpanded: true,
    hint: Text(context.tr.country),
    value: filterState.country,
    items: [
      const DropdownMenuItem(value: "", child: Text('All')),
      for (final Country country in state.data!)
        DropdownMenuItem(
          value: country.name,
          child: Text(country.name),
        ),
    ],
    onChanged: (String? newValue) {
      context.read<ComputerFilterCubit>().updateCountry(newValue);
    },
  );
  return DecoratedBox(
    decoration: BoxDecoration(
      color: const Color.fromARGB(255, 224, 232, 245),
      border:
        Border.all(color: context.watchTheme.data.primaryColorDark),
      borderRadius: BorderRadius.circular(10),
    ),
    child: Padding(
      padding: const EdgeInsets.only(left: 10, right: 10),

```

```

        child: DropdownButtonHideUnderline(
          child: DropdownButton<String>(
            isExpanded: true,
            hint: Text(context.tr.country),
            value: filterState.country,
            items: [
              const DropdownMenuItem(
                value: "", child: Text('All')),
              for (final Country country in state.data!)
                DropdownMenuItem(
                  value: country.name,
                  child: Text(country.name),
                ),
            ],
            focusColor: Colors.transparent,
            icon: const Icon(Icons.keyboard_arrow_down_rounded),
            iconEnabledColor:
              context.watchTheme.data.primaryColorDark,
            onChanged: (String? newValue) {
              context
                .read<ComputerFilterCubit>()
                .updateCountry(newValue);
            }
          )
        )
      } else {
        return const SizedBox.shrink();
      }
    }
  Widget _gpuManufacturersFilter(ComputerFilterState filterState) {
    return BlocBuilder<ComputerFilterCubit, ComputerFilterState>(
      builder: (BuildContext context, state) {
        return DropdownButton<GPUManufacturer>(
          isExpanded: true,
          hint: Text(context.tr.gpuManufacturers),
          value: filterState.gpuManufacturer,
          items: [
            for (final GPUManufacturer manufacturer in GPUManufacturer.values)
              DropdownMenuItem(
                value: manufacturer,
                child: Text(manufacturer.tr(context)),
              )
          ],
          onChanged: (GPUManufacturer? newValue) {
            if (newValue != null) {
              context
                .read<ComputerFilterCubit>()
                .updateGPUManufacturer(newValue);
            }
          }
        );
      }
    );
  }
  return DecoratedBox(
    decoration: BoxDecoration(
      color: const Color.fromARGB(255, 224, 232, 245),
      border:

```

```

        Border.all(color: context.watchTheme.data.primaryColorDark),
        borderRadius: BorderRadius.circular(10),
      ),
      child: Padding(
        padding: const EdgeInsets.only(left: 10, right: 10),
        child: DropdownButtonHideUnderline(
          child: DropdownButton<GPUManufacturer>(
            isExpanded: true,
            hint: Text(context.tr.gpuManufacturers),
            value: filterState.gpuManufacturer,
            items: [
              for (final GPUManufacturer manufacturer
                in GPUManufacturer.values)
                DropdownMenuItem(
                  value: manufacturer,
                  child: Text(manufacturer.tr(context)),
                )
            ],
            focusColor: Colors.transparent,
            icon: const Icon(Icons.keyboard_arrow_down_rounded),
            iconEnabledColor:
              context.watchTheme.data.primaryColorDark,
            onChanged: (GPUManufacturer? newValue) {
              if (newValue != null) {
                context
                  .read<ComputerFilterCubit>()
                  .updateGPUManufacturer(newValue);
              }
            }
          )
        ),
      ),
    );
  }

  if (state.isLoading()) {
    return const Center(child: CircularProgressIndicator());
  } else if (state.hasData()) {
    return DropdownButton<String?>(
      isExpanded: true,
      hint: Text(context.tr.city),
      value: filterState.city,
      items: [
        const DropdownMenuItem(value: "", child: Text('All')),
        for (final City city in state.data!)
          DropdownMenuItem(
            value: city.name,
            child: Text(city.name),
          )
      ],
      onChanged: (String? newValue) {
        context.read<ComputerFilterCubit>().updateCity(newValue);
      }
    );
    return DecoratedBox(

```

```

        decoration: BoxDecoration(
          color: const Color.fromARGB(255, 224, 232, 245),
          border:
            Border.all(color: context.watchTheme.data.primaryColorDark),
          borderRadius: BorderRadius.circular(10),
        ),
        child: Padding(
          padding: const EdgeInsets.only(left: 10, right: 10),
          child: DropdownButtonHideUnderline(
            child: DropdownButton<String?>(
              isExpanded: true,
              hint: Text(context.tr.city),
              value: filterState.city,
              focusColor: Colors.transparent,
              icon: const Icon(Icons.keyboard_arrow_down_rounded),
              iconEnabledColor:
                context.watchTheme.data.primaryColorDark,
              items: [
                const DropdownMenuItem(
                  value: "", child: Text('All')),
                for (final City city in state.data!)
                  DropdownMenuItem(
                    value: city.name,
                    child: Text(city.name),
                  )
              ],
              onChanged: (String? newValue) {
                context
                  .read<ComputerFilterCubit>()
                  .updateCity(newValue);
              }
            )
          )
        ) else {
          return const SizedBox.shrink();
        }
      }
    }

import 'package:flutter/material.dart';
import 'package:flutter_admin/data/extensions/theme_manager.dart';
import 'package:flutter_admin/data/extensions/translation_extension.dart';
import 'package:flutter_admin/widgets/simple_container.dart';
import 'package:flutter_animate/flutter_animate.dart';
import '../data/models/computer_filter.dart';

```

```

@override
Widget build(BuildContext context) {
  return _lastSeenButtonBuilder();
}

Widget _lastSeenButtonBuilder() {
  return Builder(
    builder: (context) {
      Widget head = _head(lastXdays, context);
    }
  );
}

```

```

        Widget body = _body(lastXdays, context);
        Widget head = _head(lastXdays, context);
        Widget body = _body(lastXdays, context);

        switch (orientation) {
            case LastSeenOrientation.vertical:
                return Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [head, body],
                );
            case LastSeenOrientation.horizontal:
                return Row(
                    mainAxisAlignment: MainAxisAlignment.start,
                    children: [
                        head,
                        const SizedBox(
                            width: 48,
                        ),
                        SizedBox(width: 150, child: body)
                    ],
                );
            default:
                return const SizedBox.shrink();
        }
    },
);

switch (orientation) {
    case LastSeenOrientation.vertical:
        return Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [head, body],
        );
    case LastSeenOrientation.horizontal:
        return Row(
            mainAxisAlignment: MainAxisAlignment.start,
            children: [
                head,
                const SizedBox(
                    width: 48,
                ),
                SizedBox(width: 150, child: body)
            ],
        );
    default:
        return const SizedBox.shrink();
}
}

Widget _head(int lastxDays, BuildContext context) {
    int? tempXdays = LastSeenXDays.oneDay.valuesAsList().contains(lastXdays)
        ? lastXdays
        : null;

```

```

return DropdownButton<LastSeenXDays>(
    isExpanded: true,
    value: LastSeenXDays.oneDay.fromInt(tempXdays),
    items: [
        for (final value in LastSeenXDays.values)
        DropdownMenuItem(
            value: value,
            child: Text(value.tr(context)),
        )
    ],
    onChanged: (LastSeenXDays? newValue) async {
        if (newValue != null) {
            if (LastSeenXDays.values.contains(newValue) &&
                newValue != LastSeenXDays.other) {
                radioCallback(newValue.value());
            } else {
                DateTime? res = await showDatePicker(
                    context: context,
                    initialDate: DateTime.now().subtract(lastXdays.days),
                    firstDate: DateTime(2023, 1, 1),
                    lastDate: DateTime.now());
                if (res != null) {
                    datePickerCallback(res);
                }
            }
        }
    },
);

return DecoratedBox(
    decoration: BoxDecoration(
        color: const Color.fromARGB(255, 224, 232, 245),
        border: Border.all(color: context.watchTheme.data.primaryColorDark),
        borderRadius: BorderRadius.circular(10),
    ),
    child: Padding(
        padding: const EdgeInsets.only(left: 10, right: 10),
        child: DropdownButtonHideUnderline(
            child: DropdownButton<LastSeenXDays>(
                focusColor: Colors.transparent,
                icon: const Icon(Icons.keyboard_arrow_down_rounded),
                iconEnabledColor: context.watchTheme.data.primaryColorDark,
                value: LastSeenXDays.oneDay.fromInt(tempXdays),
                items: [
                    for (final value in LastSeenXDays.values)
                    DropdownMenuItem(
                        value: value,
                        child: Text(value.tr(context)),
                    )
                ],
                onChanged: (LastSeenXDays? newValue) async {
                    if (newValue != null) {
                        if (LastSeenXDays.values.contains(newValue) &&
                            newValue != LastSeenXDays.other) {
                            radioCallback(newValue.value());
                        } else {
                            DateTime? res = await showDatePicker(

```

